

Cơ sở dữ liệu

(Ngôn ngữ SQL)

Giảng Viên: Nguyễn Thanh Bình

tháng 9, 2019



“Data are becoming a new raw material of business.”

– Craig Mundie, Microsoft

Giới thiệu



- Là ngôn ngữ để truy xuất và thao tác trên CSDL
- SQL là viết tắt của Structured Query Language
- Khởi nguồn của SQL là SEQUEL - Structured English Query Language, năm 1974)
- SQL trở thành chuẩn của ANSI vào năm 1986, và ISO năm 1987.
- Từ khóa trong SQL không phân biệt hoa thường, select và SELECT là như nhau.

Giới thiệu



- Mặc dù là một chuẩn của ANSI/ISO, có nhiều phiên bản của ngôn ngữ SQL
- Mỗi một CSDL hỗ trợ những thuộc tính mở rộng, các kiểu dữ liệu thêm vào so với chuẩn SQL.
- Tuy nhiên tối thiểu, chúng đều hỗ trợ những câu lệnh cơ bản như SELECT, UPDATE, DELETE, INSERT, WHERE với cùng một ngữ nghĩa như nhau.
- Trong khóa học này chúng ta tập trung vào **MS SQL server**
- Tham khảo thêm cú pháp tại:
 - <https://www.w3schools.com/sql/default.asp>
 - <https://www.tutorialspoint.com/sql/index.htm>

Giới thiệu



- PL SQL - "Procedural Language extensions to SQL." Là phần mở rộng của SQL được sử dụng trong Oracle.
 - Một ngôn ngữ lập trình tự nhiên dễ dàng pha trộn với SQL
 - Hỗ trợ các khái niệm như OOP
 - Insert into phải được sử dụng, toán tử MINUS dùng với Select
- T-SQL - "Transact-SQL." Là phần mở rộng của SQL được sử dụng trong Microsoft SQL Server.
 - Cho lập trình viên sự chủ động hơn khi làm việc với csdl
 - Cho phép chèn nhiều dòng vào một bảng dùng BULK INSERT
 - Select into được sử dụng, mệnh đề not exist dùng với Select

Giới thiệu

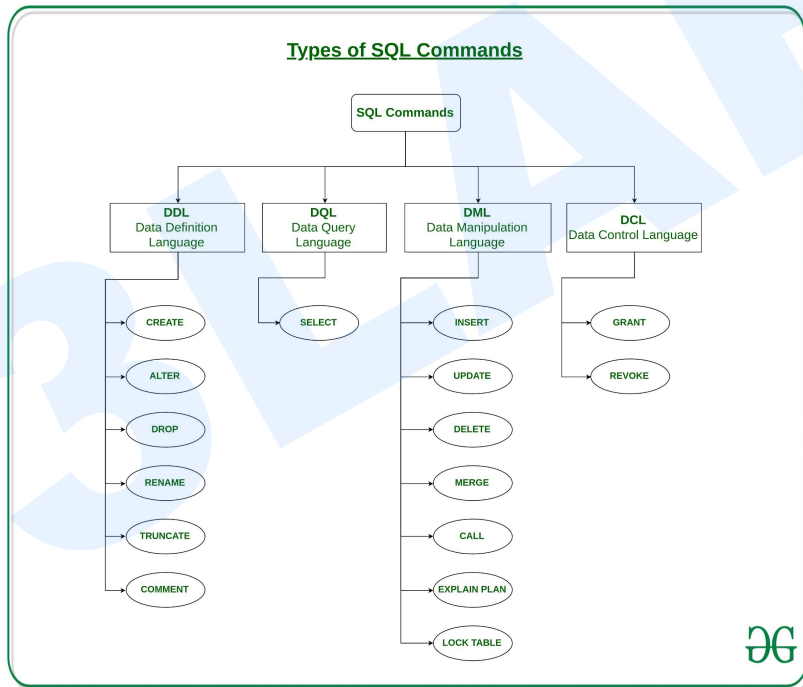


Bao gồm 4 nhóm cú pháp cơ bản

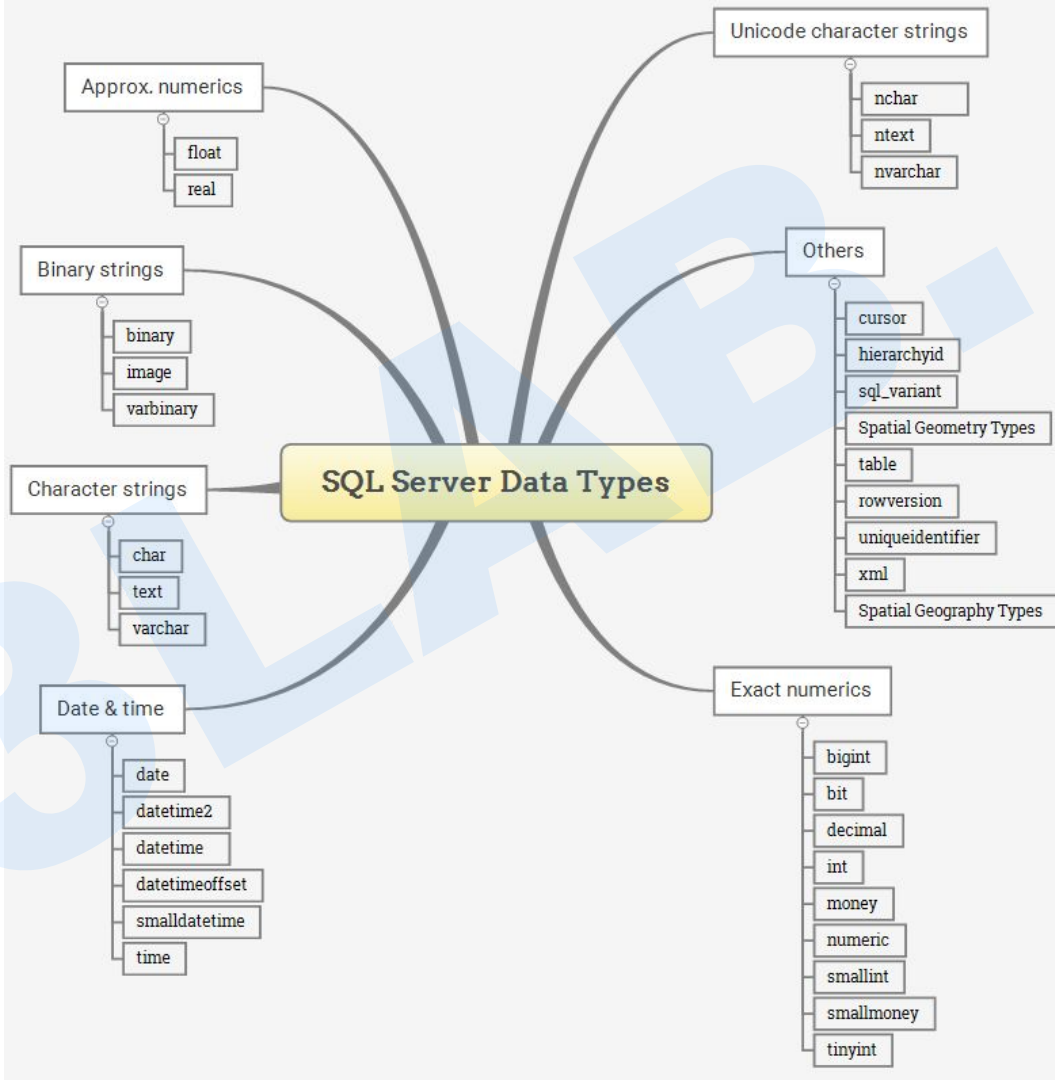
- **Ngôn ngữ định nghĩa dữ liệu** (Data Definition Language - **DDL**): cho phép khai báo cấu trúc bảng, các mối quan hệ và các ràng buộc. (create, alter, ...)
- **Ngôn ngữ thao tác dữ liệu** (Data Manipulation Language - **DML**): cho phép thêm, xóa, sửa dữ liệu. (insert, delete, update, ...)
- **Ngôn ngữ truy vấn dữ liệu** (Data Query Language – **DQL**): cho phép truy vấn dữ liệu. (Select)
- **Ngôn ngữ điều khiển dữ liệu** (Data Control Language – **DCL**): khai báo bảo mật, cấp quyền và thu hồi quyền khai thác trên cơ sở dữ liệu. (Grant, ...)

Giới thiệu

Bao gồm 4 nhóm cú pháp cơ bản



Kiểu dữ liệu



Kiểu dữ liệu



- Không phải tất cả các kiểu dữ liệu đều được hỗ trợ bởi mọi hệ cơ sở dữ liệu: **Oracle không hỗ trợ DATETIME.**
- Danh sách các kiểu dữ liệu được giới thiệu không bao gồm tất cả mà chỉ có những kiểu dữ liệu thông dụng. Mỗi hệ CSDL đôi khi có những kiểu dữ liệu riêng biệt. Ví dụ: **MS-SQL** có kiểu *money* và *smallmoney*.
- Mỗi hệ csdl có **domain** khác nhau cho các kiểu dữ liệu. Vì vậy việc nắm bắt ngữ cảnh sử dụng của từng kiểu dữ liệu quan trọng hơn là nhớ cụ thể miền giá trị của chúng.

Kiểu dữ liệu

Số (Numeric)

Data Type	Lower limit	Upper limit	Memory
bigint	-2^{63} (-9,223,372,036,854,775,808)	$2^{63}-1$ (-9,223,372,036,854,775,807)	8 bytes
int	-2^{31} (-2,147,483,648)	$2^{31}-1$ (-2,147,483,647)	4 bytes
smallint	-2^{15} (-32,767)	2^{15} (-32,768)	2 bytes
tinyint	0	255	1 byte
bit	0	1	1 byte/8bit column
decimal	$-10^{38}+1$	$10^{38}-1$	5 to 17 bytes
numeric	$-10^{38}+1$	$10^{38}-1$	5 to 17 bytes
money	-922,337,203,685,477.5808	+922,337,203,685,477.5807	8 bytes
smallmoney	-214,478.3648	+214,478.3647	4 bytes

Kiểu dữ liệu

Số thực



Data Type	Lower limit	Upper limit	Memory	Precision
float(n)	-1.79E+308	1.79E+308	Depends on the value of n	7 Digit
real	-3.40E+38	3.40E+38	4 bytes	15 Digit

Kiểu dữ liệu

Ngày Tháng

Data Type	Storage size	Accuracy	Lower Range	Upper Range
datetime	8 bytes	Rounded to increments of .000, .003, .007	1753-01-01	9999-12-31
smalldatetime	4 bytes, fixed	1 minute	1900-01-01	2079-06-06
date	3 bytes, fixed	1 day	0001-01-01	9999-12-31
time	5 bytes	100 nanoseconds	00:00:00.0000000	23:59:59.9999999
datetimeoffset	10 bytes	100 nanoseconds	0001-01-01	9999-12-31
datetime2	6 bytes	100 nanoseconds	0001-01-01	9999-12-31

Kiểu dữ liệu

Chuỗi và Ký tự



Data Type	Lower limit	Upper limit	Memory
char	0 chars	8000 chars	n bytes
varchar	0 chars	8000 chars	n bytes + 2 bytes
varchar (max)	0 chars	2^{31} chars	n bytes + 2 bytes
text	0 chars	2,147,483,647 chars	n bytes + 4 bytes

Kiểu dữ liệu

Chuỗi và Ký tự Unicode



Data Type	Lower limit	Upper limit	Memory
nchar	0 chars	4000 chars	2 times n bytes
nvarchar	0 chars	4000 chars	2 times n bytes + 2 bytes
ntext	0 chars	1,073,741,823 char	2 times the string length

Kiểu dữ liệu

Chuỗi Nhị phân



Data Type	Lower limit	Upper limit	Memory
binary	0 bytes	8000 bytes	n bytes
varbinary	0 bytes	8000 bytes	The actual length of data entered + 2 bytes
image	0 bytes	2,147,483,647 bytes	

Kiểu dữ liệu

Các kiểu khác

Data Type	Description
cursor	for variables or stored procedure OUTPUT parameter that contains a reference to a cursor
rowversion	expose automatically generated, unique binary numbers within a database.
hierarchyid	represent a tree position in a tree hierarchy
uniqueidentifier	16-byte GUID
sql_variant	store values of other data types
XML	store XML data in a column, or a variable of XML type
Spatial Geometry type	represent data in a flat coordinate system.
Spatial Geography type	store ellipsoidal (round-earth) data, such as GPS latitude and longitude coordinates.
table	store a result set temporarily for processing at a later time

Ngôn ngữ định nghĩa dữ liệu (DDL)



Cơ sở dữ liệu

- Tạo cơ sở dữ liệu
 - `CREATE DATABASE databasename;`
- Xóa cơ sở dữ liệu
 - `DROP DATABASE databasename;`
- Backup cơ sở dữ liệu
 - `BACKUP DATABASE databasename`
`TO DISK = 'filepath';`
 - `BACKUP DATABASE databasename`
`TO DISK = 'filepath'`
`WITH DIFFERENTIAL;`

Ngôn ngữ định nghĩa dữ liệu (DDL)



Bảng

- Tạo bảng

- `CREATE TABLE table_name (
 column1 datatype,
 column2 datatype,
 column3 datatype,

);`

- Tạo bảng từ bảng đã tồn tại

- `CREATE TABLE new_table_name AS
 SELECT column1, column2,...
 FROM existing_table_name
 WHERE;`

- Xóa Bảng: `DROP TABLE table_name;`

- Xóa các bộ trong Bảng: `TRUNCATE TABLE table_name;`

Ngôn ngữ định nghĩa dữ liệu (DDL)



Bảng

- Thêm cột vào bảng

- `ALTER TABLE table_name
ADD column_name datatype;`

- Xóa cột khỏi bảng

- `ALTER TABLE table_name
DROP COLUMN column_name;`

- Sửa kiểu dữ liệu của cột:

- `ALTER TABLE table_name
ALTER COLUMN column_name datatype;`

Ngôn ngữ định nghĩa dữ liệu (DDL)



View

- Cú pháp

- `CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table name
WHERE condition;`
- `CREATE OR REPLACE VIEW view name AS
SELECT column1, column2, ...
FROM table name
WHERE condition;`
- `DROP VIEW view_name;`

Ngôn ngữ định nghĩa dữ liệu (DDL)



View

- Ví dụ

- `CREATE VIEW [Products Above Average Price] AS
SELECT ProductName, Price
FROM Products
WHERE Price > (SELECT AVG(Price) FROM Products);`
- `CREATE OR REPLACE VIEW [Brazil Customers] AS
SELECT CustomerName, ContactName, City
FROM Customers
WHERE Country = "Brazil";`
- `DROP VIEW [Brazil Customers];`

Ngôn ngữ định nghĩa dữ liệu (DDL)



Ràng buộc

- Dùng để chỉ định các luật cho dữ liệu trong bảng
- Ràng buộc có thể có nhiều cấp độ, cho cột và cho bảng
- Các loại ràng buộc
 - NOT NULL - Đảm bảo cột không có giá trị NULL
 - UNIQUE - Đảm bảo tất cả các giá trị trong một cột là khác nhau
 - PRIMARY KEY - NOT NULL và UNIQUE.
 - FOREIGN KEY - Chỉ định ánh xạ đến dòng dữ liệu duy nhất trong bảng khác
 - CHECK - Chắc chắn rằng tất cả các giá trị trong cột thỏa điều kiện chỉ định
 - DEFAULT - Thiết lập giá trị mặc định cho một cột khi không cung cấp giá trị cho nó
 - INDEX - Được sử dụng để tạo và truy xuất dữ liệu từ csdl nhanh hơn

Ngôn ngữ định nghĩa dữ liệu (DDL)



Ràng buộc

- Áp dụng cho các cột riêng lẻ, dùng khi tạo create hay thay đổi alter bảng

- ```
CREATE TABLE table_name (
 column1 datatype constraint,
 column2 datatype constraint,
 column3 datatype constraint,

);
```

# Ngôn ngữ định nghĩa dữ liệu (DDL)



## *Ràng buộc - Not Null*

- Ví dụ

- `CREATE TABLE Persons (  
    ID int NOT NULL,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255) NOT NULL,  
    Age int  
);`
- `ALTER TABLE Persons  
MODIFY Age int NOT NULL;`

# Ngôn ngữ định nghĩa dữ liệu (DDL)



## *Ràng buộc - Unique*

- Ví dụ

- ```
CREATE TABLE Persons (  
    ID int NOT NULL UNIQUE,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255),  
    Age int  
);
```
- ```
CREATE TABLE Persons (
 ID int NOT NULL,
 LastName varchar(255) NOT NULL,
 FirstName varchar(255),
 Age int,
 CONSTRAINT UC_Person UNIQUE (ID, LastName)
);
```

# Ngôn ngữ định nghĩa dữ liệu (DDL)



## *Ràng buộc - Unique*

- Ví dụ

- `ALTER TABLE Persons  
ADD UNIQUE (ID);`
- `ALTER TABLE Persons  
ADD CONSTRAINT UC_Person UNIQUE (ID, LastName);`
- `ALTER TABLE Persons  
DROP CONSTRAINT UC_Person;`

# Ngôn ngữ định nghĩa dữ liệu (DDL)



## *Ràng buộc - Primary key*

- Ví dụ

- ```
CREATE TABLE Persons (  
    ID int NOT NULL PRIMARY KEY,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255),  
    Age int  
);
```
- ```
CREATE TABLE Persons (
 ID int NOT NULL,
 LastName varchar(255) NOT NULL,
 FirstName varchar(255),
 Age int,
 CONSTRAINT PK_Person PRIMARY KEY (ID, LastName)
);
```

# Ngôn ngữ định nghĩa dữ liệu (DDL)



## *Ràng buộc - Primary key*

- Ví dụ

- `ALTER TABLE Persons  
ADD PRIMARY KEY (ID);`
- `ALTER TABLE Persons  
ADD CONSTRAINT PK_Person PRIMARY KEY (ID, LastName);`
- `ALTER TABLE Persons  
DROP CONSTRAINT PK_Person;`

# Ngôn ngữ định nghĩa dữ liệu (DDL)

## Ràng buộc - Foreign key

- Ví dụ

- ```
CREATE TABLE Orders (  
    OrderID int NOT NULL PRIMARY KEY,  
    OrderNumber int NOT NULL,  
    PersonID int FOREIGN KEY REFERENCES Persons(PersonID)  
);
```
- ```
CREATE TABLE Orders (
 OrderID int NOT NULL,
 OrderNumber int NOT NULL,
 PersonID int,
 PRIMARY KEY (OrderID),
 CONSTRAINT FK_PersonOrder FOREIGN KEY (PersonID)
 REFERENCES Persons(PersonID)
);
```

# Ngôn ngữ định nghĩa dữ liệu (DDL)



## *Ràng buộc - Foreign key*

- Ví dụ

- `ALTER TABLE Orders  
ADD FOREIGN KEY (PersonID) REFERENCES Persons(PersonID);`
- `ALTER TABLE Orders  
ADD CONSTRAINT FK_PersonOrder  
FOREIGN KEY (PersonID) REFERENCES Persons(PersonID);`
- `ALTER TABLE Orders  
DROP CONSTRAINT FK_PersonOrder;`

# Ngôn ngữ định nghĩa dữ liệu (DDL)



## Ràng buộc - Check

- Ví dụ

- ```
CREATE TABLE Persons (  
    ID int NOT NULL,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255),  
    Age int CHECK (Age>=18)  
);
```
- ```
CREATE TABLE Persons (
 ID int NOT NULL,
 LastName varchar(255) NOT NULL,
 FirstName varchar(255),
 Age int,
 City varchar(255),
 CONSTRAINT CHK_Person CHECK (Age>=18 AND City='Sandnes')
);
```

# Ngôn ngữ định nghĩa dữ liệu (DDL)



## Ràng buộc - Check

- Ví dụ

- `ALTER TABLE Persons  
ADD CHECK (Age>=18);`
- `ALTER TABLE Persons  
ADD CONSTRAINT CHK_PersonAge CHECK (Age>=18 AND City='Sandnes');`
- `ALTER TABLE Persons  
DROP CONSTRAINT CHK_PersonAge;`

# Ngôn ngữ định nghĩa dữ liệu (DDL)



## *Ràng buộc - Default*

- Ví dụ

- ```
CREATE TABLE Persons (  
    ID int NOT NULL,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255),  
    Age int,  
    City varchar(255) DEFAULT 'Sandnes'  
);
```
- ```
CREATE TABLE Orders (
 ID int NOT NULL,
 OrderNumber int NOT NULL,
 OrderDate date DEFAULT GETDATE()
);
```

# Ngôn ngữ định nghĩa dữ liệu (DDL)



## *Ràng buộc - Default*

- Ví dụ

- `ALTER TABLE Persons  
ADD CONSTRAINT df_City  
DEFAULT 'Sandnes' FOR City;`
- `ALTER TABLE Persons  
ALTER COLUMN City DROP DEFAULT;`

# Ngôn ngữ định nghĩa dữ liệu (DDL)



## *Ràng buộc - Index*

- **Cú pháp**

- `CREATE INDEX index_name  
ON table_name (column1, column2, ...);`
- `CREATE UNIQUE INDEX index_name  
ON table_name (column1, column2, ...);`
- `DROP INDEX table_name.index_name;`

# Ngôn ngữ định nghĩa dữ liệu (DDL)



## *Ràng buộc - Auto Increment*

- Cú pháp

- `CREATE TABLE Persons (  
    Personid int IDENTITY(1,1) PRIMARY KEY,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255),  
    Age int  
);`
- Giá trị bắt đầu = 1 và mỗi lần tăng 1 giá trị

# Ngôn ngữ thao tác dữ liệu (DML)



- Gồm các lệnh liên quan tới dòng dữ liệu (tuple, row, record):
  - Lệnh thêm dữ liệu (INSERT)
  - Lệnh sửa dữ liệu (UPDATE)
  - Lệnh xóa dữ liệu (DELETE)

# Ngôn ngữ thao tác dữ liệu (DML)



## *Insert*

- Cú pháp
  - INSERT INTO tên\_bảng (cột1,...,cộtn) VALUES (giá\_trị\_1,..., giá\_trị\_n)
  - INSERT INTO tên\_bảng VALUES (giá\_trị\_1, giá\_trị\_2,..., giá\_trị\_n)
  - SELECT \* INTO tên-bảng-mới from tên-bảng-có-sẵn
  - INSERT INTO tên-bảng-tạo-trước select \* from tên-bảng-có-sẵn
- Ví dụ
  - insert into SANPHAM values('BC01','But chi', 'cay', 'Singapore', 3000)
  - insert into SANPHAM(masp,tensp,dvt,nuocsx,gia) values ('BC01','But chi','cay','Singapore',3000)

# Ngôn ngữ thao tác dữ liệu (DML)



## *Insert*

- Ví dụ của select...into
  - `Select * into SANPHAM_NEW from SANPHAM`
  - `Select * into SANPHAM_NEW from SANPHAM WHERE điều-kiện`
- Ví dụ của insert into....select....
  - `insert into SANPHAM_COPY select * from SANPHAM`
  - `insert into SANPHAM_COPY select * from SANPHAM where điều-kiện`

# Ngôn ngữ thao tác dữ liệu (DML)



## *Update*

- Cú pháp
  - UPDATE tên\_bảng  
SET cột\_1 = giá\_trị\_1, cột\_2 = giá\_trị\_2 ....  
[WHERE điều\_kiện]
- Lưu ý: lưu ý với các lệnh xóa và sửa, nếu không có điều kiện ở WHERE nghĩa là xóa hoặc sửa tất cả.
- Ví dụ: Tăng giá 10% đối với những sản phẩm do “Trung Quoc” sản xuất
  - UPDATE SANPHAM  
SET Gia = Gia\*1.1  
WHERE Nuocsx='Trung Quoc'

# Ngôn ngữ thao tác dữ liệu (DML)



## *Delete*

- Cú pháp
  - DELETE FROM tên\_bảng [WHERE điều\_kiện]
- Ví dụ:
  - Xóa toàn bộ nhân viên  
DELETE FROM NHANVIEN
  - Xóa những sản phẩm do Trung Quốc sản xuất có giá thấp hơn 10000  
DELETE FROM SANPHAM  
WHERE (Gia < 10000) and (Nuocsx = 'Trung Quoc')

# Ngôn ngữ truy vấn (DQL)



- Dùng truy vấn dữ liệu từ cơ sở dữ liệu
- Cú pháp tổng quát
  - `SELECT` [`DISTINCT`, `TOP`] danh\_sách\_cột | hàm
  - `FROM` danh sách các quan hệ (hay bảng, table)
  - [`WHERE` điều\_kiện]
  - [`GROUP BY` danh\_sách\_cột\_gom\_nhóm]
  - [`HAVING` điều\_kiện\_trên\_nhóm]
  - [`ORDER BY` cột1 ASC | DESC, cột2 ASC | DESC, ... ]

# Ngôn ngữ truy vấn (DQL)



## *Các toán tử (Operators)*

- Toán tử so sánh:
  - =, >, <, >=, <=, <>
- Toán tử logic: AND, OR, BETWEEN, IS NULL / IS NOT NULL, LIKE (% , \_), IN / NOT IN, EXISTS / NOT EXISTS, SOME, ALL, ANY.
- Các phép toán số học: +, - , \* , /, %
- Phép toán hỗn hợp: +=, -=, \*=, /=, %=, &=, ^=, |=
- Các phép toán Bit: &, |, ^
- Các hàm xử lý ngày (DAY( )), tháng (MONTH( )), năm (YEAR( ))

# Ngôn ngữ truy vấn (DQL)



- 5 hàm: **COUNT( )**, **SUM( )**, **MAX( )**, **MIN( )**, **AVG( )**
- Phân loại câu **SELECT**:
  - **SELECT** đơn giản,
  - **SELECT** có mệnh đề **ORDER BY**,
  - **SELECT** lồng (câu **SELECT** lồng câu **SELECT** khác),
  - **SELECT** gom nhóm (**GROUP BY**),
  - **SELECT** gom nhóm (**GROUP BY**) có điều kiện **HAVING**

# Ngôn ngữ truy vấn (DQL)



*Ví dụ*

- Cho lược đồ CSDL “quản lý đề án công ty” như sau

NHANVIEN (MaNV, HoTen, Phai, Luong, NTNS, Ma\_NQL, MaPH)

PHONGBAN (MaPH, TenPH, TRPH)

DEAN (MaDA, TenDA, Phong, NamThucHien)

PHANCONG (MaNV, MaDA, ThoiGian)

# Ngôn ngữ truy vấn (DQL)

Ví dụ

| MANV | HOTEN             | NTNS       | PHAI | MA_NQL | MaPH | LUONG     | NHANVIEN |
|------|-------------------|------------|------|--------|------|-----------|----------|
| 001  | Vuong Ngoc Quyen  | 22/10/1957 | Nu   |        | QL   | 3.000.000 |          |
| 002  | Nguyen Thanh Tung | 09/01/1955 | Nam  | 001    | NC   | 2.500.000 |          |
| 003  | Le Thi Nhan       | 18/12/1960 | Nu   | 001    | DH   | 2.500.000 |          |
| 004  | Dinh Ba Tien      | 09/01/1968 | Nam  | 002    | NC   | 2.200.000 |          |
| 005  | Bui Thuy Vu       | 19/07/1972 | Nam  | 003    | DH   | 2.200.000 |          |
| 006  | Nguyen Manh Hung  | 15/09/1973 | Nam  | 002    | NC   | 2.000.000 |          |
| 007  | Tran Thanh Tam    | 31/07/1975 | Nu   | 002    | NC   | 2.200.000 |          |
| 008  | Tran Hong Minh    | 04/07/1976 | Nu   | 004    | NC   | 1.800.000 |          |

## DEAN

| MADA  | TENDA         | PHONG | NamThucHien |
|-------|---------------|-------|-------------|
| TH001 | Tin hoc hoa 1 | NC    | 2002        |
| TH002 | Tin hoc hoa 2 | NC    | 2003        |
| DT001 | Dao tao 1     | DH    | 2004        |
| DT002 | Dao tao 2     | DH    | 2004        |

## PHONGBAN

| MAPH | TENPH      | TRPH |
|------|------------|------|
| QL   | Quan Ly    | 001  |
| DH   | Dieu Hanh  | 003  |
| NC   | Nghien Cuu | 002  |

## PHANCONG

| MANV | MADA  | THOIGIAN |
|------|-------|----------|
| 001  | TH001 | 30,0     |
| 001  | TH002 | 12,5     |
| 002  | TH001 | 10,0     |
| 002  | TH002 | 10,0     |
| 002  | DT001 | 10,0     |
| 002  | DT002 | 10,0     |
| 003  | TH001 | 37,5     |
| 004  | DT001 | 22,5     |
| 004  | DT002 | 10,0     |
| 006  | DT001 | 30,5     |
| 007  | TH001 | 20,0     |
| 007  | TH002 | 10,0     |
| 008  | DT002 | 12,5     |

# Ngôn ngữ truy vấn (DQL)



Ví dụ - *BETWEEN & ORDER BY*

- Câu hỏi 1: Sử dụng **=, >, >=, ...** Danh sách các nhân viên sinh trong khoảng từ năm 1978 đến 1983?
  - **Select** MaNV, HoTen **From** NhanVien  
**where** Year(NTNS) **>=** 1978 **AND** Year(NTNS) **<=** 1983
- Câu hỏi 2: Sử dụng **BETWEEN, ORDER BY**. Danh sách các nhân viên sinh trong khoảng từ năm 1978 đến 1983? Sắp xếp theo mức lương giảm dần.
  - **Select** \* **From** NhanVien **where** Year(NTNS) **BETWEEN** 1978 **and** 1983 **ORDER BY** Luong **DESC**

# Ngôn ngữ truy vấn (DQL)



Ví dụ - *IS NULL & IS NOT NULL*

- Câu hỏi 3: Sử dụng **IS NULL**. Cho biết những nhân viên không có người quản lý trực tiếp? (không chịu sự quản lý trực tiếp của người nào)
  - **Select** MaNV, HoTen, NTNS, Ma\_NQL **from** NhanVien **where** Ma\_NQL **is Null**
- Câu hỏi 4: Sử dụng **Is Not Null**. Cho biết những nhân viên có người quản lý trực tiếp. Thông tin hiển thị gồm: mã nhân viên, họ tên, mã người quản lý.
  - **Select** MaNV, HoTen, Ma\_NQL **from** NhanVien **where** Ma\_NQL **is not Null**

# Ngôn ngữ truy vấn (DQL)

Ví dụ - *IN & NOT IN*

- Câu hỏi 5: Sử dụng **IN** (so sánh với một tập hợp giá trị cụ thể). Cho biết họ tên nhân viên thuộc phòng 'NC' hoặc phòng 'DH'?
  - **Select DISTINCT** Hoten **From** NhanVien **where** MaPH **in** ('NC','DH')
- Câu hỏi 6: Sử dụng **IN** (so sánh với một tập hợp giá trị chọn từ câu **SELECT** khác). Cho biết họ tên nhân viên thuộc phòng 'NC' hoặc phòng 'DH'?.
  - **Select** Hoten **from** NhanVien **where** MaPH **in** (**Select** MaPH **from** PHONGBAN **where** MaPH='NC' **OR** MaPH='DH')

# Ngôn ngữ truy vấn (DQL)



Ví dụ - *IN & NOT IN*

- Câu hỏi 7: Cho biết mã số, họ tên, ngày tháng năm sinh của những nhân viên đã tham gia đề án?
  - **Select** MaNV, HoTen, NTNS **from** NhanVien **where** MaNV **in** (**Select** MaNV **From** PhanCong)
- Câu hỏi 8: Sử dụng **NOT IN**. Cho biết mã số, họ tên, ngày tháng năm sinh của những nhân viên không tham gia đề án nào? Gợi ý cho mệnh đề **NOT IN**: thực hiện câu truy vấn “tìm nhân viên có tham gia đề án (dựa vào bảng PhanCong)”, sau đó lấy phần bù.

# Ngôn ngữ truy vấn (DQL)



Ví dụ - IN & NOT IN

- Câu hỏi 8:
  - **Select** MaNV, HoTen, NTNS **from** NhanVien **where** MaNV **not in** (**Select** MaNv **From** PhanCong)
- Câu hỏi 9: Cho biết tên phòng ban không chủ trì các đề án triển khai năm 2005? Gợi ý: thực hiện câu truy vấn “tìm phòng ban chủ trì các đề án triển khai năm 2005”, sau đó lấy phần bù.
  - **Select** TenPH **from** PhongBan **where** MaPH **not in** (**Select** **DISTINCT** Phong **from** DEAN **where** NamThucHien=2005)

# Ngôn ngữ truy vấn (DQL)



Ví dụ - *EXISTS* & *NOT EXISTS*

- Câu hỏi 10: tiếp tục câu hỏi số 8, nhưng trả lời với Not Exists
  - **Select \* from NhanVien as nv where not exists (Select \* From PhanCong as pc Where pc.MaNV = nv.MaNV)**
- Câu hỏi 11: Tiếp tục câu hỏi số 7, nhưng trả lời bằng Exists.
  - **Select \* from NhanVien as nv where exists (Select \* From PhanCong as pc Where pc.MaNV = nv.MaNV)**

# Ngôn ngữ truy vấn (DQL)

Ví dụ - *EXISTS & NOT EXISTS*

- Câu hỏi 12: tìm danh sách nhân viên tham gia tất cả các đề án
  - **Select distinct \* from NhanVien nv Where Not Exists(  
Select \* from Dean as da where not exists (  
Select \* From PhanCong as pc  
Where pc.MaNV = nv.MaNV and pc.MaDA = da.MaDA)  
)**

# Ngôn ngữ truy vấn (DQL)



*Ví dụ - So sánh LIKE*

- Câu hỏi 13: so sánh chuỗi = chuỗi. Liệt kê mã nhân viên, ngày tháng năm sinh, mức lương của nhân viên có tên “Nguyễn Tường Linh”?
  - **Select** MaNV, NTNS, Luong **from** NhanVien **where** HoTen = ‘Nguyễn Tường Linh’
- Câu hỏi 14: Sử dụng **LIKE** (%: thay thế 1 chuỗi ký tự). Tìm những nhân viên có họ Nguyễn.
  - **Select** MaNV, HoTen **from** NhanVien **where** HoTen **like** ‘Nguyễn %’

# Ngôn ngữ truy vấn (DQL)



*Ví dụ - So sánh LIKE*

- Câu hỏi 15: Tìm những nhân viên có tên Lan
  - **Select** MaNV, HoTen **from** NhanVien **where** HoTen **like** '% Lan'
- Câu hỏi 16: Tìm những nhân viên có tên lót là "Văn".
  - **Select** MaNV, HoTen **from** NhanVien **where** HoTen **like** '% Văn %'
- Câu hỏi 17: Sử dụng **LIKE** ( \_: thay thế 1 ký tự bất kỳ). Tìm những nhân viên tên có tên 'Nguyễn La\_' (ví dụ Lam, Lan)
  - **Select** MaNV, HoTen **from** NhanVien **where** HoTen **like** 'Nguyễn La\_'

# Ngôn ngữ truy vấn (DQL)



*Các hàm Count, Sum, Min, Max, Avg trên toàn bộ quan hệ và Ví dụ*

- Câu hỏi 18: Tính số nhân viên của công ty
  - **Select COUNT(MaNV) as SoNV from NhanVien**
- Câu hỏi 19: Tính số lượng nhân viên quản lý trực tiếp nhân viên khác.
  - **Select COUNT(DISTINCT Ma\_NQL) from NhanVien**
- Câu hỏi 20: Tìm mức lương lớn nhất, mức lương trung bình, tổng lương của công ty.
  - **Select MAX(Luong), AVG(Luong), SUM(Luong) from NhanVien**

# Ngôn ngữ truy vấn (DQL)



*Các hàm Count, Sum, Min, Max, Avg trên toàn bộ quan hệ và Ví dụ*

- Câu hỏi 21: Cho biết nhân viên có mức lương lớn nhất.
  - **Select** HoTen **from** NhanVien **Where** Luong = (**Select MAX**(Luong) **from** NhanVien )
- Câu hỏi 22: Cho biết nhân viên có mức lương trên mức lương trung bình của công ty.
  - **Select** HoTen **from** NhanVien **where** Luong > (**Select AVG**(Luong) **from** NhanVien )

# Ngôn ngữ truy vấn (DQL)



*Count, Sum, Min, Max, AVG trên từng nhóm nhỏ (GROUP BY)*

- Chia các dòng thành các nhóm nhỏ dựa trên tập thuộc tính chia nhóm.
- Thực hiện các phép toán trên nhóm như: Count (thực hiện phép đếm), Sum (tính tổng), Min(lấy giá trị nhỏ nhất), Max(lấy giá trị lớn nhất), AVG (lấy giá trị trung bình).

# Ngôn ngữ truy vấn (DQL)

*Count, Sum, Min, Max, AVG trên từng nhóm nhỏ (GROUP BY)*

Quan hệ NV

| Q | S  |
|---|----|
| a | 10 |
| a | 2  |
| b | 9  |
| b | 5  |
| c | 10 |
| c | 8  |
| c | 6  |
| c | 4  |
| c | 10 |
| d | 16 |
| d | 18 |
| d | 50 |

nhóm

Chia các dòng thành các nhóm dựa trên tập thuộc tính chia nhóm

| Q | Count(S) |
|---|----------|
| a | 2        |
| b | 2        |
| c | 5        |
| d | 3        |

Tương tự cho các hàm SUM, MIN, MAX, AVG

Các thuộc tính GROUP BY: Q

Câu SQL:

```
Select Q, count(S)
From NV
Group by Q
```

# Ngôn ngữ truy vấn (DQL)



## Mệnh đề Group By và Ví dụ

- Câu hỏi 23: Cho biết số lượng nhân viên theo từng phái?
  - Do cột phái có 2 giá trị “nam” và “nữ”, trường hợp này ta chia bảng NhanVien thành 2 nhóm nhỏ. Thuộc tính chia nhóm là thuộc tính “Phai”.
  - **Select** Phai, **count**(Manv) **as** SoNV **from** NhanVien **Group by** Phai
- Câu hỏi 24: Cho biết số lượng nhân viên theo từng phòng?
  - Do cột MaPH có 3 giá trị “NC” và “DH” và “QL”, trường hợp này ta chia bảng nhân viên thành 3 nhóm nhỏ. Thuộc tính chia nhóm là thuộc tính “MaPH”.
  - **Select** MaPH, **count**(Manv) **from** NhanVien **Group by** MaPH
  - Tương tự: cho biết tổng lương của mỗi phòng, cho biết mức lương thấp nhất của từng phòng, mức lương cao nhất, mức lương trung bình của từng phòng

# Ngôn ngữ truy vấn (DQL)



## Mệnh đề Group By và Ví dụ

- Câu hỏi 25: Cho biết tên phòng và số lượng nhân viên theo từng phòng?
  - Giống câu 21 nhưng bổ sung thêm bảng PhongBan để lấy tên phòng. Thuộc tính chia nhóm là (TenPH) thay cho MaPH.
  - **Select** TenPH, **count**(Manv) **as** SoLuongNV **From** NhanVien n, PhongBan p **Where** n.MaPh=p.MaPH **Group by** TenPH
- Câu hỏi 26: Với mỗi phòng, cho biết số lượng nhân viên theo từng phái?
  - Do cột MaPH có 3 giá trị “NC” và “DH” và “QL”, mỗi phòng chia nhỏ theo từng phái: 2 nhóm “Nam” và “Nữ”, trường hợp này ta chia bảng nhân viên thành 6 nhóm nhỏ. Như vậy, tập thuộc tính chia nhóm cho câu truy vấn là (Phong, Phai).
  - **Select** MaPH, Phai, **count**(Manv) **from** NhanVien **Group by** Phong, Phai

# Ngôn ngữ truy vấn (DQL)



## Mệnh đề Group By và Ví dụ

- Câu hỏi 27: Đếm số đề án của từng nhân viên tham gia?
  - Do cột MaNV có 7 giá trị “NV001”,...”NV008” (không có nhân viên “005”), trường hợp này ta chia bảng PhanCong thành 7 nhóm nhỏ. Với mỗi nhóm nhỏ (MaNV), ta đếm số đề án (count(MADA)) tham gia. Thuộc tính chia nhóm là thuộc tính “MaNV”.
  - **Select** MaNV, **count**(MaDA) **as** SoDATG **From** PhanCong **Group by** MaNV
  - Tương tự: tính tổng số giờ làm việc của mỗi nhân viên (SUM), thời gian làm việc thấp nhất của mỗi nhân viên (MIN), thời gian làm việc lớn nhất của mỗi nhân viên (MAX), thời gian làm việc trung bình,...
- Câu hỏi 28: Cho biết mã, tên nhân viên và số đề án mà n/v đã tham gia?
  - **Select** n.MaNV, HoTen, count(MaDA) **as** SoDATG **From** PhanCong pc, NhanVien n **where** pc.manv=n.manv **Group by** MaNV, HoTen

# Ngôn ngữ truy vấn (DQL)



## Mệnh đề Having và Ví dụ

- Lọc kết quả theo điều kiện, sau khi đã gom nhóm
- Điều kiện của **HAVING** là điều kiện về các hàm tính toán trên nhóm (**Count, Sum, Min, Max, AVG**) và các thuộc tính trong danh sách **GROUP BY**.
- Câu hỏi 29: Cho biết những nhân viên tham gia từ 2 đề án trở lên?
  - **Select** MaNV, **count**(MaDA) **as** SoDATG **From** PhanCong **Group by** MaNV **Having** **count**(MaDA) **>=** 2
- Câu hỏi 30: Cho biết mã phòng ban có trên 4 nhân viên?
  - **Select** MaPH, **count**(Manv) **from** NhanVien **Group by** MaPH **Having** **count**(Manv) **>** 4

# Ngôn ngữ truy vấn (DQL)



*Ví dụ với ANY và ALL*

- ANY: Trả ra TRUE nếu bất kỳ giá trị nào trong truy vấn con thỏa điều kiện.
- ALL: Trả ra TRUE nếu tất cả giá trị trong truy vấn con thỏa điều kiện.
- Cú pháp:
  - SELECT \*  
FROM NHANVIEN  
WHERE MaNV = **ANY** (SELECT MaNV FROM PHANCONG WHERE Thoigian < 10);
  - SELECT \*  
FROM NHANVIEN  
WHERE MaNV = **ALL** (SELECT MaNV FROM PHANCONG WHERE Thoigian < 10);

# Ngôn ngữ truy vấn (DQL)



*Ví dụ với UNION và UNION ALL*

- UNION: mặc định chỉ lấy các bộ giá trị distinct.
- UNION ALL: giống UNION nhưng chấp nhận trùng.
- Cú pháp:

- `SELECT column_name(s) FROM table1`

**UNION | UNION ALL**

`SELECT column_name(s) FROM table2;`

# Ngôn ngữ truy vấn (DQL)



*Ví dụ với INTERSECT*

- Trả về các bộ chung từ hai câu select.
- Cú pháp:
  - `SELECT column1 [, column2 ]`  
`FROM table1 [, table2 ]`  
`[WHERE condition]`

## **INTERSECT**

```
SELECT column1 [, column2]
FROM table1 [, table2]
[WHERE condition]
```

# Ngôn ngữ truy vấn (DQL)



*Ví dụ với EXCEPT*

- Trả về các bộ thuộc câu select thứ nhất mà không thuộc câu select thứ hai.
- Cú pháp:
  - `SELECT column1 [, column2 ]  
FROM table1 [, table2 ]  
[WHERE condition]`

## **EXCEPT**

```
SELECT column1 [, column2]
FROM table1 [, table2]
[WHERE condition]
```

# Ngôn ngữ truy vấn (DQL)



## SQL JOINS

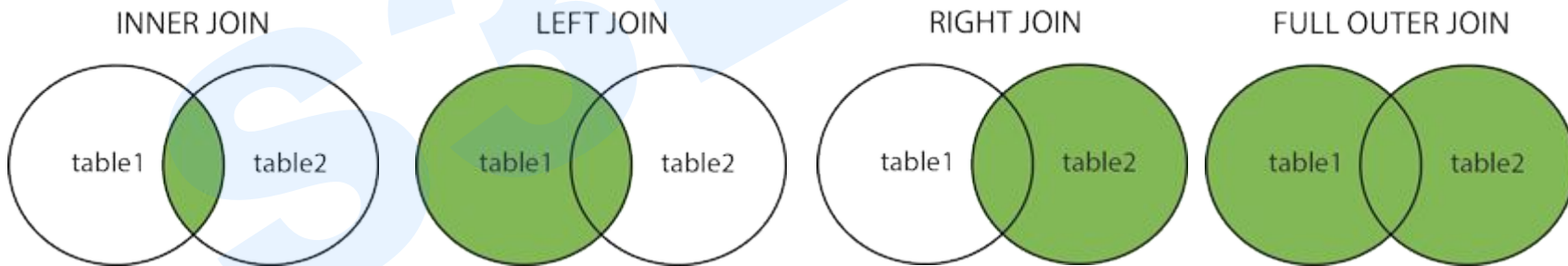
- (INNER) JOIN: Trả về các bộ có các giá trị liên kết ở cả hai bảng
- LEFT (OUTER) JOIN: Trả về tất cả các bộ từ bảng bên trái, và các bộ liên kết từ bảng bên phải
- RIGHT (OUTER) JOIN: Trả về tất cả các bộ từ bảng bên phải, và các bộ liên kết từ bảng bên trái
- FULL (OUTER) JOIN: Trả về tất cả các bộ dù có hay không có liên kết của bảng bên trái cũng như bảng bên phải

# Ngôn ngữ truy vấn (DQL)

## SQL JOINS

- Cú pháp:

- `SELECT column_name(s)`  
`FROM table1`  
`[INNER | LEFT | RIGHT | FULL OUTER] JOIN table2`  
`ON table1.column_name = table2.column_name;`



# Ngôn ngữ truy vấn (DQL)



## *Self JOIN*

- Cú pháp:
  - `SELECT column_name(s)`  
`FROM table1 t1, table1 t2`  
`WHERE condition;`

# Ngôn ngữ điều khiển (DCL)



## *grant*

- Được sử dụng để gán quyền truy cập cho user vào một database
- Cú pháp:
  - `GRANT SELECT, UPDATE ON MY_TABLE TO SOME_USER, ANOTHER_USER;`

# Ngôn ngữ điều khiển (DCL)



## *revoke*

- Được sử dụng để lấy lại quyền hạn từ một user
- Cú pháp:
  - `REVOKE SELECT, UPDATE ON MY_TABLE FROM USER1, USER2;`



## Cảm ơn đã theo dõi

Chúng tôi hy vọng cùng nhau đi đến thành công.